
Out-of-Distribution Detection for Hardware Verification Properties Using Deep Representation Learning

Gabriel Martin¹

Abstract

Formal hardware verification engineers frequently reuse SystemVerilog Assertions across modules, risking silent property-design incompatibilities that waste debugging effort. We frame property-design mismatch detection as an out-of-distribution detection problem over paired RTL modules and SVA properties. Our model combines AST-derived RTL features, a BiLSTM SVA encoder, a four-way interaction head, and energy-bounded learning. On a small benchmark of 22 RTL modules and 296 SVA properties, energy-bounded learning improves leave-one-module-out energy AUROC from 0.634 to 0.676, while a less restrictive random-pair split reaches 0.758. Identifier anonymization and modality ablations show that joint RTL-SVA modeling is necessary, but the high LOO FPR@95%TPR and small dataset indicate that generalization to unseen module types remains the main limitation.

1. Introduction

Ensuring the functional correctness of digital hardware is one of the most resource-intensive activities in semiconductor design. Assertion-based verification (ABV) has become a cornerstone methodology: engineers write formal properties, most commonly in SystemVerilog Assertions (SVA), that describe invariants, safety conditions, and protocol sequences a design must satisfy. A formal property checker then mathematically proves or disproves these assertions against the register-transfer level (RTL) model.

In industrial practice, verification engineers can frequently reuse SVA properties across different modules or design revisions. While reuse accelerates development, it introduces a serious risk: a property correctly specifying one module may be silently incompatible with another, producing false failures, difficult proof attempts, or misleading passes. Diagnosing such mismatches is costly, particularly when the incompatibility is non-obvious from port or signal names.

This work proposes *automatically detecting mismatched property-design pairs before running formal verification*.

We frame the task as an OOD detection problem: given a set of correct property-module pairs (in-distribution), we train a deep representation learning model to recognize when a property has been cross-applied to a structurally incompatible module (out-of-distribution). The central question of this work is whether a model trained on known-compatible RTL-SVA pairs can identify mismatched property-design pairs for modules not seen during training. This setting is stricter than ordinary pair classification because the model must avoid memorizing module-specific naming patterns or property templates and instead learn structural patterns that transfer across designs. Our contributions are:

- A joint RTL-SVA compatibility model combining 18-dimensional AST-derived RTL features, a BiLSTM SVA encoder, L2-normalized interaction features, and a composite cross-entropy, cosine, and energy-bounded training objective.
- A small but structured benchmark of 22 RTL modules and 296 SVA properties for studying property-design mismatch detection under synthetic cross-module OOD pairs.
- A leave-one-module-out evaluation showing that energy-bounded learning improves energy AUROC from 0.634 to 0.676, while high FPR@95%TPR shows that generalization remains limited in the small-data regime.
- Identifier-anonymization and modality ablations showing that joint RTL-SVA modeling is necessary and that suppressing name leakage improves energy-score separability.

2. Related Work

Machine Learning for Hardware Verification. [Wu et al. \(2024\)](#) provide a comprehensive survey of ML techniques applied across the hardware verification lifecycle, covering simulation-based testing, formal methods, and coverage closure (ensuring all design behaviors have been tested). They identify the automation of property generation and verification as open directions, motivating approaches that can validate or screen generated properties.

Automated Assertion Generation. Witharana et al. (2022) propose an automated framework for generating security assertions for RTL models using static code analysis and execution trace analysis across six classes of SoC vulnerabilities. While rule-based rather than learning-based, their work demonstrates that structural analysis of RTL designs can be systematically mapped to formal property templates, supporting the premise that structure–property relationships can be automated and motivating learned approaches to the same problem.

RTL Semantic Representations. Vasudevan et al. (2021) introduce Design2Vec, a deep representation learning framework that constructs semantic embeddings of RTL designs from control-flow and data-flow graph structures, supporting downstream tasks such as coverage prediction and test generation. Our approach is most closely related in spirit, but uses lightweight AST-derived scalar and ratio features rather than full graph embeddings, making the method practical with limited training data. We further extend the framework to the joint property-design compatibility problem.

Energy-Based OOD Detection. Liu et al. (2020) propose energy-based scoring as a principled mechanism for OOD detection, deriving a scalar energy score from classifier logits without requiring density estimation or generative models. In-distribution inputs produce systematically lower (more negative) energy than OOD inputs. Beyond using energy scores at inference time, we adopt their energy-bounded training objective, which directly penalizes in-distribution samples with insufficiently low energy and OOD samples with insufficiently high energy, explicitly optimizing the energy gap during training.

These directions motivate the present work but do not directly address paired RTL-SVA compatibility: determining whether a specific formal property is appropriate for a specific RTL module. Assertion-generation methods produce candidate properties, RTL representation methods usually model designs alone, and generic OOD methods require task-specific representations. This work combines these ideas in a joint RTL-SVA mismatch detector.

3. Technical Approach

Our pipeline consists of five stages: (1) RTL preprocessing and feature extraction, (2) identifier anonymization, (3) SVA property encoding via BiLSTM, (4) joint compatibility prediction via a four-way interaction head, and (5) OOD detection using energy-based scoring at inference time.

Each example is a pair (M, p) of an RTL module and SVA property. Pairs are labeled compatible/in-distribution when p was written for M , and incompatible/OOD when p is cross-applied to another module. The task is therefore pairwise RTL-SVA compatibility, not standalone module or

property classification.

3.1. RTL Preprocessing and Feature Extraction

RTL modules written in SystemVerilog (.sv) are first converted to plain Verilog (.v) using `sv2v` (Snow, 2020), normalizing syntax before parsing. The converted file is then parsed into an Abstract Syntax Tree using PyVerilog. If PyVerilog parsing fails, the system automatically falls back to a regex-based extractor that operates on comment-stripped source text. Both paths produce the same feature representation.

From the AST or source text, we extract an 18-dimensional structural feature vector:

$$\mathbf{f}(M) = [n_{\text{in}}, n_{\text{out}}, n_{\text{reg}}, n_{\text{wire}}, n_{\text{seq}}, n_{\text{comb}}, n_{\text{ctrl}}, n_{\text{ops}}, r_{\text{in}}, r_{\text{seq}}, r_{\text{reg}}, n_{\text{params}}, n_{\text{gen}}, n_{\text{fn}}, n_{\text{case}}, b_{\text{clk}}, n_{\text{lines}}, \rho_{\text{ctrl}}] \in \mathbb{R}^{18} \quad (1)$$

The vector contains port, register, wire, sequential/combinational block, control-flow, operation, parameter, generate, function, and case-statement counts, along with derived ratios, a clock-presence indicator, source-line count, and control-flow density. These features capture high-level structural constraints relevant to property applicability, such as whether a module contains clocked behavior, storage, arbitration logic, or explicit control flow. In the LOO setting, feature vectors are standardized using a `StandardScaler` fitted only on training modules to prevent leakage.

3.2. Identifier Anonymization

A key concern is *name leakage*: a model may learn to match signal names shared between a property and a module rather than structural compatibility. We suppress this by applying identifier anonymization before training and evaluation.

Within each module, all signal, register, wire, and parameter identifiers are replaced with canonical (placeholder) tokens, while a fixed SVA/Verilog keyword blocklist prevents reserved language tokens from being anonymized. Interface signals are mapped to namespace $\{\text{IN}_0, \text{IN}_1, \dots, \text{OUT}_0, \text{OUT}_1, \dots\}$, preserving port directionality; internal signals map to $\{\text{ID}_0, \text{ID}_1, \dots\}$. The RTL anonymization map is built first and then applied identically to all SVA properties of that module. The impact is quantified in Section 4.7.

3.3. SVA Property Encoding

Each anonymized SVA property is tokenized by a custom tokenizer that resolves SVA multi-character operators ($| \rightarrow$, $| = >$, $[*$, $[=$, $[- >$, $\#\#$) before falling back to iden-

tifier, numeric, and single-character rules. A shared vocabulary is built over all properties; each is encoded as a $\langle \text{BOS} \rangle / \langle \text{EOS} \rangle$ -padded sequence of length $L_{\max} = 128$.

A BiLSTM with hidden size H per direction processes the token sequence, producing output matrix $\mathbf{O} \in \mathbb{R}^{T \times 2H}$. A validity mask $\mathbf{m} \in \{0, 1\}^T$ zeroes padding positions. Three pooling operations are applied:

$$\mathbf{p}_{\text{mean}} = \frac{\sum_t m_t \mathbf{O}_t}{\sum_t m_t} \quad (2)$$

$$\mathbf{p}_{\text{max}} = \max_{t: m_t=1} \mathbf{O}_t \quad (3)$$

$$\mathbf{p}_{\text{attn}} = \sum_t \alpha_t \mathbf{O}_t, \quad \alpha_t \propto \exp(a_t) \quad (4)$$

where $a_t = \mathbf{w}_2^\top \tanh(\mathbf{W}_1 \mathbf{O}_t)$ (masked to $-\infty$ at padding). The three vectors are concatenated and projected to the property embedding:

$$\mathbf{e}_p = \mathbf{W}_{\text{proj}} [\mathbf{p}_{\text{mean}}; \mathbf{p}_{\text{max}}; \mathbf{p}_{\text{attn}}] \in \mathbb{R}^H \quad (5)$$

3.4. RTL Feature Encoder

The 18-dimensional feature vector is passed through a two-layer MLP with input Layer Normalization and GELU activations:

$$\mathbf{e}_M = \text{GELU}(\mathbf{W}_2 \text{GELU}(\mathbf{W}_1 \text{LN}(\mathbf{f}(M)))) \in \mathbb{R}^{d_r} \quad (6)$$

with dropout applied after the first activation.

3.5. Joint Compatibility Classifier

Both embeddings are projected to a shared matching space $\mathbb{R}^{d_m}$ and L2-normalized:

$$\hat{\mathbf{e}}_M = \frac{\mathbf{W}_r \mathbf{e}_M}{\|\mathbf{W}_r \mathbf{e}_M\|_2}, \quad \hat{\mathbf{e}}_p = \frac{\mathbf{W}_s \mathbf{e}_p}{\|\mathbf{W}_s \mathbf{e}_p\|_2} \quad (7)$$

A four-way interaction vector is then formed:

$$\mathbf{u} = [\hat{\mathbf{e}}_M; \hat{\mathbf{e}}_p; |\hat{\mathbf{e}}_M - \hat{\mathbf{e}}_p|; \hat{\mathbf{e}}_M \odot \hat{\mathbf{e}}_p] \in \mathbb{R}^{4d_m} \quad (8)$$

capturing each embedding individually, along with element-wise differences and similarities. This is passed through a three-layer classifier φ :

$$\mathbf{z} = \varphi(\mathbf{u}) \in \mathbb{R}^2 \quad (9)$$

where φ applies $\text{LN} \rightarrow \text{Linear}(4d_m, d_j) \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(d_j, d_j/2) \rightarrow \text{GELU} \rightarrow \text{Dropout} \rightarrow \text{Linear}(d_j/2, 2)$.

3.6. Composite Training Objective

Training uses three loss terms. **Cross-entropy with label smoothing** ϵ :

$$\mathcal{L}_{\text{CE}} = - \sum_{(M,p,y) \in \mathcal{D}} \sum_{c \in \{0,1\}} \tilde{y}_c \log q_c, \quad q_c = \frac{\exp(z_c)}{\sum_{j=0}^1 \exp(z_j)} \quad (10)$$

where $\tilde{y}_c = (1 - \epsilon) \mathbf{1}[c = y] + \epsilon/2$. **Cosine embedding loss** on the normalized embeddings:

$$\mathcal{L}_{\text{cos}} = \sum_{(M,p,y) \in \mathcal{D}} \ell_{\text{cos}}(\hat{\mathbf{e}}_M, \hat{\mathbf{e}}_p, y), \quad (11)$$

where, with $s = \cos(\hat{\mathbf{e}}_M, \hat{\mathbf{e}}_p)$ and margin δ ,

$$\ell_{\text{cos}} = \begin{cases} 1 - s, & y = 1, \\ \max(0, s - \delta), & y = 0. \end{cases} \quad (12)$$

Energy-bounded loss (Liu et al., 2020) that directly optimizes the energy gap between in-distribution and OOD samples:

$$\begin{aligned} \mathcal{L}_{\text{energy}} = & \sum_{\substack{(M,p,y) \in \mathcal{D} \\ y=1}} \max(0, E(M,p) - m_{\text{in}})^2 \\ & + \sum_{\substack{(M,p,y) \in \mathcal{D} \\ y=0}} \max(0, m_{\text{out}} - E(M,p))^2 \end{aligned} \quad (13)$$

where m_{in} is the upper energy bound for in-distribution inputs and m_{out} is the lower energy bound for OOD inputs. The total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{CE}} + \lambda_c \mathcal{L}_{\text{cos}} + \lambda_e \mathcal{L}_{\text{energy}} \quad (14)$$

Models are optimized with AdamW and gradient clipping.

3.7. Energy-Based OOD Detection

At inference time, the energy score (Liu et al., 2020) is computed from the logit vector $\mathbf{z}$:

$$E(M,p) = -T \cdot \log \sum_{c=0}^1 \exp\left(\frac{z_c}{T}\right) \quad (15)$$

with $T = 1$. Compatible pairs produce more negative energy; mismatched pairs produce higher energy. A pair is flagged as OOD when $E(M,p) > \gamma$, with γ tuned on a validation set.

4. Experiments

4.1. Dataset

We construct a benchmark of **22 RTL modules**: barrel shifter, clock divider, comparator, CRC generator, dual-port RAM, edge detector, FIFO buffer, fixed-latency pipeline, FSM controller, handshake controller, leading-zero counter, multiplexer tree, parity unit, popcount, priority encoder, pulse stretcher, register file, round-robin arbiter, shift register, skid buffer, up-down counter, and watchdog timer. For each module, roughly 8-16 SVA properties were written covering safety invariants, protocol sequencing, and interface invariants, yielding **296 total properties** across all 22

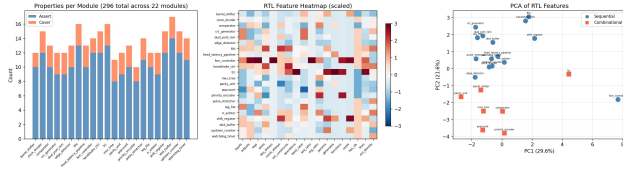


Figure 1. Dataset overview. *Left*: property counts per module (assert vs. cover). *Center*: scaled RTL feature heatmap showing structural diversity across the 22 modules. *Right*: PCA of RTL feature vectors, with sequential modules (circles) and combinational modules (squares) separating along PC1 (29.6% variance explained).

modules (Figure 1). The benchmark includes 15 sequential (clock-edge-sensitive) modules and 7 combinational modules.

All properties were grouped by source module and treated as compatible only with that module. Cross-module pairs were used as synthetic mismatches. This construction models property-reuse risk but is limited by the small benchmark size: only 22 module families and 296 properties are available, so some held-out modules have few structurally similar training examples. This small-data regime likely contributes to high per-fold variance and weak LOO generalization. It may also introduce label noise when generic properties are meaningful for multiple modules, and near-duplicate property templates were not exhaustively removed. For training OOD pairs, we use hard-negative sampling: 50% are drawn from the $k = 3$ nearest modules in RTL feature space and 50% uniformly at random.

4.2. Training Configurations

Three configurations train the same `CompatibilityClassifier` architecture under different evaluation protocols and loss settings.

Base LOO. Leave-one-out cross-validation with $\lambda_e = 0$ (cross-entropy and contrastive loss only). Early stopping minimizes validation CE loss. Per-fold `StandardScaler` is fit on training modules only.

Random Split + EBL. A single 65/15/20 pair split across all modules with the full composite loss ($\lambda_e = 0.1$, $m_{\text{in}} = -0.8$, $m_{\text{out}} = -0.4$). Early stopping maximizes validation energy AUROC. This configuration is less rigorous since training and test pairs share the same module set, but provides an upper bound on achievable performance.

LOO + EBL. Leave-one-out cross-validation with the full composite loss. Early stopping maximizes validation energy AUROC per fold. This is the most rigorous configuration.

Table 1. Aggregated OOD detection results. AUROC_E and FPR95_E use the energy score; higher AUROC and lower FPR95 are better.

CONFIGURATION	ACC	F1	AUROC _E	FPR95 _E
COIN FLIP	0.500	0.500	0.500	0.950
BASE LOO	0.644	0.692	0.634	0.885
LOO + EBL	0.642	0.625	0.676	0.905
RAND. SPLIT+EBL	0.746	0.783	0.758	0.778

4.3. Implementation Details

All models use: BiLSTM hidden size $H = 64$ per direction, embedding dimension 32, RTL encoder hidden dim $d_r = 64$, matching dim $d_m = 64$, joint classifier hidden dim $d_j = 128$. Label smoothing $\epsilon = 0.05$ (LOO) or 0.1 (random split), contrastive margin $\delta = 0.2$, $\lambda_c = 0.05$. Training uses AdamW (lr = 10^{-3} , weight decay 10^{-3}), up to 300 epochs, patience 30, gradient clipping at $\|\cdot\|_2 \leq 1.0$, batch size 16. RTL parsing uses PyVerilog 1.1.x with `sv2v` v0.0.12 for SystemVerilog preprocessing.

4.4. Evaluation Metrics

OOD detection is evaluated by AUROC and FPR@95%TPR with respect to the binary OOD label. Classification performance is measured by accuracy and F1. Two scoring functions are compared: energy score $E(M, p)$ and confidence score $-P(\text{compatible} \mid M, p)$ (negated so higher values indicate more likely OOD). A random coin-flip baseline (AUROC = 0.500, FPR@95%TPR = 0.950) provides the lower bound.

4.5. Results

Table 1 presents the aggregated results. The Random Split + EBL configuration achieves the strongest classification performance (Accuracy = 0.746, F1 = 0.783) and energy AUROC of 0.758 (FPR@95%TPR = 0.778), above the coin-flip baseline. However, because training and test pairs share the same module set, it measures generalization within the same set of modules rather than true transfer to unseen module types.

The LOO configurations are more rigorous. LOO + EBL improves over Base LOO on energy AUROC (0.676 vs. 0.634, Figure 2), demonstrating that energy-bounded training provides measurable benefit even under the strict held-out module protocol. However, the high FPR@95%TPR of 0.905 shows that the current model is not yet reliable enough for deployment as an automatic verification filter. The main conclusion is therefore not that the task is solved, but that energy-bounded representation learning provides a measurable starting point for property-design mismatch detection. Notably, in both LOO settings the confidence scorer outperforms the energy scorer (Base LOO: AUROC_C = 0.689

Table 2. Per-fold energy AUROC for Base LOO and LOO+EBL across all 22 held-out modules.

MODULE	BASE LOO	LOO+EBL
BARREL_SHIFTER	0.236	0.819
CLOCK_DIVIDER	0.724	0.787
COMPARATOR	0.888	0.864
CRC_GENERATOR	0.653	0.681
DUAL_PORT_RAM	0.750	0.826
EDGE_DETECTOR	0.728	0.734
FIFO	0.606	0.664
FIXED_LAT_PIPELINE	0.302	0.450
FSM_CONTROLLER	0.265	0.117
HANDSHAKE_CTRL	0.484	0.756
LZC	0.164	0.492
MUX_TREE	0.860	0.909
PARITY_UNIT	0.882	0.931
POPCOUNT	0.953	0.935
PRIORITY_ENCODER	0.920	0.960
PULSE_STRETCHER	0.684	0.607
REG_FILE	0.882	0.646
RR_ARBITER	0.715	0.806
SHIFT_REGISTER	0.840	0.351
SKID_BUFFER	0.723	0.689
UPDOWN_COUNTER	0.569	0.631
WATCHDOG_TIMER	0.633	0.633
MEAN	0.634	0.676

vs. $\text{AUROC}_E = 0.634$; LOO+EBL: $\text{AUROC}_C = 0.696$ vs. $\text{AUROC}_E = 0.676$), suggesting that energy-bounded training is particularly important for energy scoring to realize its advantage, a pattern visible in the ROC curves of Figure 3.

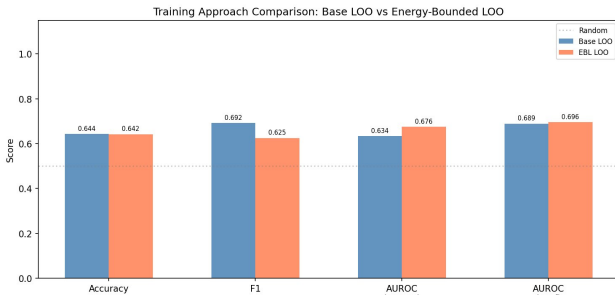


Figure 2. Comparison of Base LOO and LOO + EBL across Accuracy, F1, energy AUROC, and confidence AUROC. EBL improves energy AUROC by 0.042 while the other metrics remain comparable.

Table 2 shows that per-fold performance varies substantially. Structurally distinctive modules, `priority_encoder` (0.960), `parity_unit` (0.931), `mux_tree` (0.909), yield the highest LOO+EBL AUROCs. The hardest cases, `fsm_controller` (0.117) and `lzc` (0.492), have the largest minimum feature-space distances to their nearest neighbors (Figure 4, left), consistent with their structural signatures being atypical yet not distinctive enough to sepa-

rate.

Figure 3 shows the energy and confidence score distributions for the Base LOO model aggregated across all 22 folds ($n = 296$ ID, $n = 296$ OOD). The energy gap is -0.351 (OOD samples produce higher raw energy) and the confidence gap is 0.187 (ID samples score higher $P(\text{compatible})$). While both distributions overlap substantially, the confidence separator is cleaner, consistent with its higher AUROC.

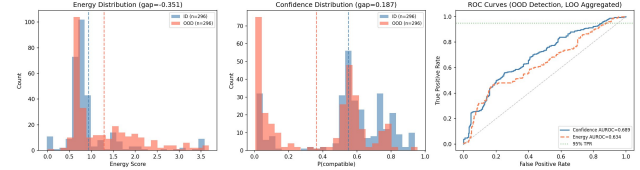


Figure 3. Energy and confidence distributions (left, center) and ROC curves (right) for the Base LOO model aggregated over all 22 LOO folds. Confidence scoring ($\text{AUROC} = 0.689$, solid blue) outperforms energy scoring ($\text{AUROC} = 0.634$, dashed orange) in the base setting.

4.6. Error Analysis

Figure 4 analyzes per-module difficulty. Accuracy decreases as each held-out module becomes more isolated in RTL feature space (Pearson $r = -0.52$), suggesting that the small training set lacks enough nearby module families for reliable transfer. Combinational modules are also easier than sequential ones (mean Acc/AUROC = $0.75/0.70$ vs. $0.61/0.64$), likely because sequential designs share clocks, registers, and always blocks that make modules harder to distinguish.

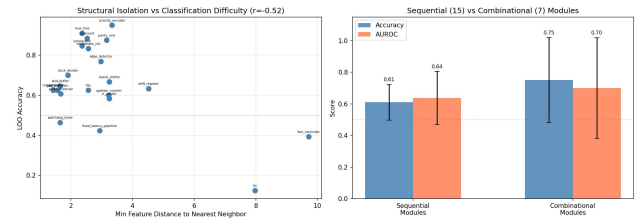


Figure 4. Left: LOO accuracy vs. structural isolation (Pearson $r = -0.52$). Modules far from their nearest neighbor in feature space (`lzc`, `fsm_controller`) are the hardest to classify correctly. Right: Combinational modules (7) achieve higher mean accuracy and AUROC than sequential modules (15).

4.7. Ablation Studies

Identifier Anonymization. Table 3 and Figure 5 show that anonymization leaves accuracy unchanged (0.662) but improves F1 (0.699 vs. 0.649) and energy AUROC (0.656

Table 3. Identifier anonymization ablation (LOO). Anonymization improves energy AUROC by 0.033 and F1 by 0.050, and lowers FPR@95%TPR (energy) by 0.084, while accuracy is identical.

IDENTIFIERS	ACC	F1	AUROC _C	AUROC _E	FPR95 _C	FPR95 _E
ANONYMIZED	0.662	0.699	0.720	0.656	0.878	0.841
ORIGINAL	0.662	0.649	0.725	0.623	0.868	0.926

Table 4. Modality ablation. RTL-only collapses to near-random performance; the joint model is necessary for meaningful detection.

MODEL	ACC	F1	AUROC
JOINT (RTL+SVA)	0.747	0.771	0.827
SVA-ONLY	0.527	0.293	0.545
RTL-ONLY	0.500	0.086	0.503

vs. 0.623). Confidence AUROC is nearly tied, so the main benefit appears to be cleaner separation in the energy scores rather than uniformly better classification. This supports the use of anonymization to reduce name-matching shortcuts in the small-data setting.

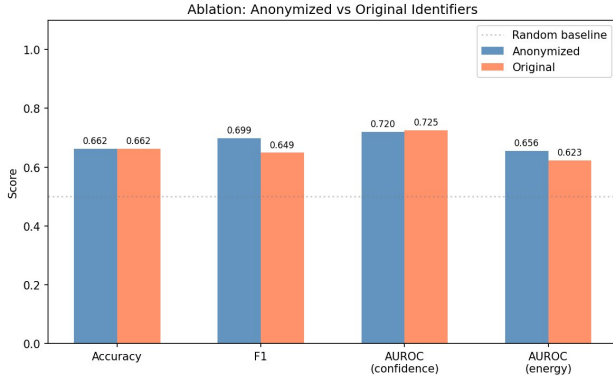


Figure 5. Anonymization ablation. Anonymized identifiers improve F1 by 0.050 and energy AUROC by 0.033 relative to original identifiers, while confidence AUROC and accuracy remain comparable.

Modality Ablation. Table 4 and Figure 6 compare joint, RTL-only, and SVA-only variants under the random-pair split. RTL-only collapses to chance performance (AUROC = 0.503), and SVA-only remains weak (AUROC = 0.545). The joint model reaches AUROC = 0.827, confirming that compatibility detection requires both module structure and property text.

5. Conclusion and Future Work

We proposed a deep representation learning framework for detecting mismatched RTL-SVA pairs in assertion-based hardware verification. The model combines AST-derived RTL features, a BiLSTM SVA encoder, four-way interaction

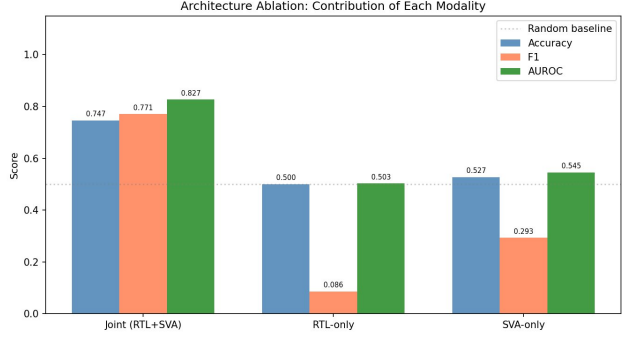


Figure 6. Modality ablation results. RTL-only achieves near-random F1 (0.086) and AUROC (0.503), demonstrating that the SVA encoder is essential. SVA-only similarly fails to reach meaningful performance. Only the joint model achieves strong detection (AUROC = 0.827).

features, and energy-bounded learning to estimate property-design compatibility.

On a small 22-module, 296-property benchmark, LOO+EBL improves energy AUROC from 0.634 to 0.676. However, the high LOO+EBL FPR@95%TPR of 0.905 and large per-fold variance show that the dataset is too small to support reliable transfer to unseen module families. The current system should therefore be viewed as a preliminary proof of concept rather than a deployable verification filter.

Several directions remain open. The RTL representation could be enriched by replacing the scalar AST feature vector with a graph neural network over the control-flow or data-flow graph, following the spirit of Design2Vec (Vasudevan et al., 2021). A Transformer-based property encoder with pre-training on larger SVA datasets could replace the BiLSTM once more data is available. The OOD framework could be extended to produce ranked structural explanations for detected mismatches, supporting interpretable verification debugging. Finally, integration with open-source formal tools such as SymbiYosys would enable evaluation on realistic, industry-scale verification runs.

Team Contribution Statement

This project was completed individually by Gabriel Martin as a solo team member. All contributions listed below were performed by Gabriel Martin.

Project ideation and design. Framed SVA-RTL compatibility checking as an OOD detection task, designed the benchmark, and selected the main technical components.

Code development. Implemented the full Python/PyTorch pipeline, including RTL preprocessing, AST/regex feature extraction, identifier anonymization, SVA tokenization, the BiLSTM and RTL encoders, the interaction classifier, and

all training configurations.

Experiments and analysis. Ran the main experiments, ablations, and error analysis, and produced the report figures.

Presentation slides. Prepared the project presentation materials.

Use of External Code and AI Tools, including LLMs

AI tools. Claude (Anthropic) was used as an editing, debugging, and review assistant for report polishing, mathematical notation, PyTorch debugging, metric checking, and code review. All model architecture decisions, experimental designs, and scientific interpretations were made by the author.

External libraries. The following open-source libraries were used and are gratefully acknowledged: PyTorch <https://pytorch.org>, scikit-learn <https://scikit-learn.org>, PyVerilog <https://github.com/PyHDI/Pyverilog>, sv2v <https://github.com/zachjs/sv2v>, NumPy <https://numpy.org>, and Matplotlib <https://matplotlib.org>.

RTL module sources. The 22 RTL modules used in the benchmark were drawn from publicly available open-source repositories, primarily OpenCores (<https://opencores.org>) and GitHub. Modules that could not be found open-source were written from scratch by the author. No proprietary or restricted RTL was used.

Code and Repository

Project code is provided as a self-contained Google Colab notebook (EEL6812 Final Project.ipynb) in the submitted [GitHub repository](#). The repository also includes the RTL source files and SVA property files. Model checkpoints are omitted, but the notebook includes instructions for reproducing them.

References

- Liu, W., Wang, X., Owens, J. D., and Li, Y. Energy-based out-of-distribution detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pp. 21464–21475, 2020. URL <https://proceedings.neurips.cc/paper/2020/file/f5496252609c43eb8a3d147ab9b9c006-Paper.pdf>.
- Snow, Z. sv2v: SystemVerilog to Verilog, 2020. URL <https://github.com/zachjs/sv2v>. Version 0.0.12.
- Vasudevan, S., Jiang, W., Bieber, D., Singh, R., Shojaei, H.,

Ho, C. R., and Sutton, C. A. Learning semantic representations to verify hardware designs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, 2021. URL https://proceedings.neurips.cc/paper_files/paper/2021/file/c5aa65949d20f6b20e1a922c13d974e7-Paper.pdf.

Witharana, H., Jayasena, A., Whigham, A., and Mishra, P. Automated generation of security assertions for RTL models. *ACM Journal on Emerging Technologies in Computing Systems*, 19(1), 2022. doi: 10.1145/3565801.

Wu, N., Li, Y., Yang, H., Chen, H., Dai, S., Hao, C., Yu, C., and Xie, Y. Survey of machine learning for software-assisted hardware design verification: Past, present, and prospect. *ACM Transactions on Design Automation of Electronic Systems*, 29(3), 2024. doi: 10.1145/3661308.